Infra Waste Metrics: the arithmetic behind the chart

Memory, energy and water per unit of served work for seven stack profiles, with every printed figure the conclusion of a machine-checked theorem

Abstract

The infographic *Infra Waste Metrics (Lean-Verified)* prints seven bar values, one spread factor and three percentages. This note states the model those figures come from, records the parameters they are computed from, and names the theorem that proves each one. All arithmetic is exact rational arithmetic; no figure is a measurement of any operator's fleet.

1 What is claimed, and what is not

Claimed. Given the parameter set of Section 3, the figures of Section 5 follow by the definitions of Section 2, and the derived quantities move in the directions listed in Section 6 when the parameters move. Each statement is a theorem in a Lean 4 development that compiles without `sorry` and uses only the standard axioms `propext`, `Classical.choice` and `Quot.sound`.

Not claimed. No parameter in Section 3 is an operator measurement. Node counts, resident working sets, idle draw, site water usage effectiveness and request volume are not published for these services. The seven profiles are constructed to be representative of an *architecture* – a managed-runtime store at replication factor three with a cache tier, a shard-per-core native store, an edge estate, a tropical colocation – and are labelled with the product each architecture is publicly associated with. They are not descriptions of any company's infrastructure. The model layer contains no number and no name: replacing the parameters changes every figure below and no identity or comparative static.

2 The model

Definition 1 (Stack)**.** A stack is the tuple

$$(N,\ R,\ D,\ \rho,\ \omega,\ W_{\text{idle}},\ W_{\text{peak}},\ u,\ P,\ (\lambda_{\text{site}}, \lambda_{\text{grid}}),\ Q,\ c)$$

of rationals: node count N; RAM per node R (GB); distinct useful resident state D (GB, one copy, no runtime overhead); resident copies ρ; runtime multiplier ω (bytes on the heap per byte of data); idle and peak node draw $W_{\text{idle}}, W_{\text{peak}}$ (W); utilization u, the fraction of the dynamic range in use; power usage effectiveness P; water intensities on site and in generation $\lambda_{\text{site}}, \lambda_{\text{grid}}$ (L/kWh); served work Q (millions of requests per month); electricity price c ($/kWh).

Definition 2 (Derived quantities)**.** Write $H = 730$ for hours in an average month.

$$\begin{aligned}
\text{provisioned} &= NR, & \text{resident} &= D\rho, & \text{working set} &= D\rho\omega,\\
\text{headroom} &= NR - D\rho\omega, & \text{wasted RAM} &= NR - D, & m &= \tfrac{NR-D}{NR},\\
\text{node draw } W &= W_{\text{idle}} + u\,(W_{\text{peak}} - W_{\text{idle}}), & \text{IT draw } I &= NW, & \text{meter } F &= IP,\\
\text{dynamic} &= Nu(W_{\text{peak}} - W_{\text{idle}}), & \text{idle} &= NW_{\text{idle}}, & \text{cooling} &= I(P-1),
\end{aligned}$$

waste power $V = \text{idle} + \text{cooling}$, energy waste fraction $e = V/F$, monthly energy $FH/1000$ kWh, wasted energy $VH/1000$ kWh, litres $= \text{kWh} \cdot (\lambda_{\text{site}} + \lambda_{\text{grid}})$, and the headline metric

$$L \;=\; \frac{1}{Q} \cdot \frac{VH}{1000}\,(\lambda_{\text{site}} + \lambda_{\text{grid}}) \qquad \text{(wasted litres per million requests)}.$$

Three identities hold for every stack, with no side conditions:

$$NR = D + \underbrace{D(\rho - 1)}_{\text{copies}} + \underbrace{D\rho(\omega - 1)}_{\text{runtime}} + \underbrace{NR - D\rho\omega}_{\text{headroom}}, \qquad (\texttt{provisioned\_decomposition})$$

$$F = \text{dynamic} + \text{idle} + \text{cooling}, \qquad (\texttt{facilityPower\_decomposition})$$

$$\text{wasted litres} = \text{wasted kWh} \cdot (\lambda_{\text{site}} + \lambda_{\text{grid}}). \qquad (\texttt{wastedLitres\_eq})$$

The first says the gap between provisioned RAM and useful state is exactly three things. The second says the meter is exactly the work, the idling and the building. The third says water waste is not an independent quantity: it is wasted energy times two intensities, one of which survives air cooling (`wastedLitres_dry`).

Every division above carries a positivity hypothesis, collected in a well-formedness predicate ($N, R, D, W_{\text{idle}}, Q > 0$; $\rho, \omega, P \geq 1$; $0 \leq u \leq 1$; $W_{\text{idle}} \leq W_{\text{peak}}$; both intensities and the price nonnegative; and the working set fits in the provisioned RAM). Each of the seven profiles and each variant is proved well formed before any of its figures is quoted.

3 The parameters

These are assumptions. Water intensities are the pair $(\lambda_{\text{site}}, \lambda_{\text{grid}})$ in litres per kWh.

profile	N	R	D	ρ	ω	W_{idle}	W_{peak}	u	P	Q	$(\lambda_{\text{site}}, \lambda_{\text{grid}})$
Cloudflare / Pingora	20000	128	500000	4	1.15	100	350	0.45	1.06	2000000	(0.05, 1.8)
Discord / ScyllaDB	1800	512	250000	3	1.15	180	600	0.55	1.15	400000	(0.3, 1.8)
Spotify / GKE	12000	128	300000	2	1.6	120	400	0.35	1.1	600000	(0.2, 1.8)
Shopify / MySQL	8000	256	400000	3	1.5	150	500	0.3	1.12	300000	(0.25, 1.8)
Netflix / Cassandra	6000	256	220000	3	2	160	520	0.4	1.15	250000	(0.35, 1.8)
Dropbox / Envoy	5000	192	300000	2	1.35	140	460	0.38	1.14	120000	(0.28, 1.8)
Grab / Kafka	3000	128	60000	3	1.8	130	430	0.32	1.4	80000	(1.8, 2.2)

Electricity is \$0.09/kWh except at the edge estate (\$0.10) and the tropical site (\$0.12); the price enters only the money line of Section 5. Three variants change exactly one thing each: the cached store on a native runtime (ω from 2 to 1.15); the scheduler fleet filled up (u from 0.35 to 0.7, and therefore Q doubled to 1200000); and the tropical log on a temperate campus (P from 1.4 to 1.1, intensities to $(0.2, 1.8)$).

4 A worked derivation

The tropical log, end to end, to show the chain the theorems automate. Node draw $130 + 0.32 \cdot 300 = 226$ W; IT draw $3000 \cdot 226 = 678000$ W; meter $678000 \cdot 1.4 = 949200$ W. Idle $3000 \cdot 130 = 390000$ W; cooling $678000 \cdot 0.4 = 271200$ W; waste 661200 W, so

$$e \;=\; \frac{661200}{949200} \;=\; 551/791 \;\approx\; 69.66\%.$$

Monthly energy $949200 \cdot 730/1000$; the wasted part is 482676 kWh (`grab_energy_month`). Water intensity is $1.8 + 2.2 = 4$ L/kWh, so wasted litres are 1930704 a month; over 80000 million requests,

$$L \;=\; 120669/5000 \;=\; 24.1338 \text{ L per million requests} \qquad (\texttt{grab\_wastedWater}),$$

of which the printed label is the two-decimal rounding (`bar_grab`). Total litres, wasted or not, are $173229/5000 \approx 34.65$ (`grab_totalWater`); the electricity bought by idling and cooling is $1448028/25 = \$57{,}921.12$ a month (`grab_wastedUSD`). On the memory side, provisioned RAM is 384000 GB against 60000 GB of distinct state, so $m = 27/32 \approx 84.38\%$ (`grab_memoryWaste`).

5 The figures on the chart

5.1 The seven bars

Wasted litres per million requests, exact and as printed. Each printed decimal is proved to be within half a unit in the last place of the exact value, i.e. $|x - d| \le 1/200$.

profile	exact L	printed	value theorem	label theorem
Cloudflare / Pingora	1218151/800000	1.52	`cloudflare_wastedWater`	`bar_cloudflare`
Discord / ScyllaDB	66680901/40000000	1.67	`discord_wastedWater`	`bar_discord`
Spotify / GKE	51757/12500	4.14	`spotify_wastedWater`	`bar_spotify`
Shopify / MySQL	900893/125000	7.21	`shopify_wastedWater`	`bar_shopify`
Netflix / Cassandra+EVCache	2420169/312500	7.74	`netflix_wastedWater`	`bar_netflix`
Dropbox / Envoy	10476011/937500	11.17	`dropbox_wastedWater`	`bar_dropbox`
Grab / Kafka	120669/5000	24.13	`grab_wastedWater`	`bar_grab`

The order the bars are drawn in is itself a theorem, stated as six strict inequalities between the exact values rather than as a sorted list of decimals (`water_ranking`), so it does not depend on how many decimals are printed.

5.2 The spread

Theorem 3 (`water_spread`, `spread_rounds`)**.** *The ratio of the largest to the smallest bar is exactly* 264480/16687*, and* $|264480/16687 - 1585/100| \le 1/200$*: the printed* 15.85× spread *is that ratio to two decimals.*

5.3 The 58% badge

Summing the seven profiles: 109909457/10 kWh a month at the meter, of which 62884317/10 kWh is idle draw and facility overhead; 11668347959/500 litres, of which 6783217159/500 litres are attributable to those wasted kilowatt-hours (`fleet_monthlyKWh`, `fleet_wastedKWh`, `fleet_monthlyLitres`, `fleet_wastedLitres`).

Theorem 4 (`fleet_water_waste_share`, `water_share_rounds`)**.** *The wasted share of the seven profiles' water is exactly* 92920783/159840383*, and this is within* 1/200 *of* 58/100*.*

For completeness, the same sum in money and memory: 596900633/1000, i.e. $596,900.63 a month of electricity bought by idling and cooling (`fleet_wastedUSD`), and 7915600 GB of provisioned RAM that is not distinct useful state (`fleet_wastedGB`).

5.4 The three levers

Theorem 5 (`native_runtime_moves_nothing`)**.** *Move the cached store to a native runtime, changing* ω *from 2 to 1.15 and nothing else. The working set falls from* 1320000 *GB to* 759000 *GB, headroom rises by exactly those* 561000 *GB, and both the memory waste fraction and the wasted litres per million requests are* unchanged*.*

This is the chart's right-hand claim: the rewrite removes heap but converts overhead into headroom, because the RAM was already provisioned and still draws power.

Theorem 6 (`packing_cuts_water`, `packing_cut_fraction`)**.** *Double utilization on the same boxes, so that they serve twice the requests. The energy waste fraction falls from* 709/1199 *to* 379/869 *(59.13% to 43.61%) and the metric from* 51757/12500 *to* 27667/12500*. The relative cut is exactly* 24090/51757*, which lies strictly between* 46/100 *and* 47/100 *— the chart prints the floor,* −46%*.*

Theorem 7 (`geography_cuts_water`, `geography_cut_fraction`)**.** *Move the tropical log to a temperate campus, changing only* P *and the two water intensities. The metric falls from* 120669/5000 *to* 167097/20000*, a difference of* 315579/20000 *litres per million requests. The relative cut is exactly* 315579/482676*, within* 1/200 *of* 65/100 *— the chart's* −65%*.*

5.5 Two order facts behind the framing

Theorem 8 (`all_waste_over_half`)**.** *Every one of the seven profiles has energy waste fraction above* $1/2$*: the smallest is the edge estate at* $451/901 \approx 50.06\%$*, the largest the tropical log at* $551/791 \approx 69.66\%$*.*

Theorem 9 (`memory_ranking`)**.** *Every one wastes more than two thirds of its provisioned RAM against distinct useful state: the smallest is the mesh at* $11/16 = 68.75\%$*, the largest the managed-runtime store with a cache tier at* $329/384 \approx 85.68\%$*. The scheduler, the relational tier and the edge estate coincide at* $103/128 \approx 80.47\%$*.*

6 Comparative statics

Proved once in the model layer, for every stack and independently of any parameter value.

lever	direction	theorem
more resident copies	less headroom	`headroom_antitone_replication`
fatter runtime	less headroom	`headroom_antitone_runtime`
more RAM for the same data	more wasted RAM	`wastedGB_mono_ram`
higher utilization	lower energy waste fraction	`energyWasteFraction_antitone_utilization`
worse PUE	more wasted watts	`wastePower_mono_pue`
more idle draw	more wasted water	`wastedLitres_mono_idle`
more requests, same fleet	fewer kWh per request	`kWhPerMillionRequests_antitone`

Two limiting statements accompany them: a fleet at zero utilization wastes its entire meter (`energyWasteFraction_eq_one_of_idle`), and the waste term vanishes only if a node draws nothing at rest and the building is free to run (`wastePower_eq_zero_iff`), which no well-formed profile satisfies.

7 Reproduction

Build with `lake build RequestProject.InfraWasteInfographic`. A consistency script checks that every exact rational quoted in this paper and in the project's prose occurs in a Lean statement, compared as rationals rather than as decimals.

file	contents
`RequestProject/Theory/InfraWaste.lean`	the model: definitions, identities, comparative statics. No number, no name.
`RequestProject/Theory/Water.lean`	the two water channels, on site and in generation.
`RequestProject/Data/PlatformStacks.lean`	the parameters of Section 3, as assumptions.
`RequestProject/InfraWasteLedger.lean`	the exact rationals and the ranking.
`RequestProject/InfraWasteInfographic.lean`	the printed decimals, the factor and the percentages.
`RequestProject/InfraWasteDemo.lean`	the same figures printed as tables.